Intelligent Robotics Final Presentation

인공지능학과 2391037 한채헌
Team: F40

https://github.com/chehun16/ROS2-2026

01. Topic

01 Topic

- Topic: Camera-Based BEV Occupancy Map Generation and Autonomous Mobile Robot System Using Depth Information
- Goal: Integrate RGB-D camera-based obstacle perception and Nav2-based autonomous navigation in indoor environments.

02. Preliminary

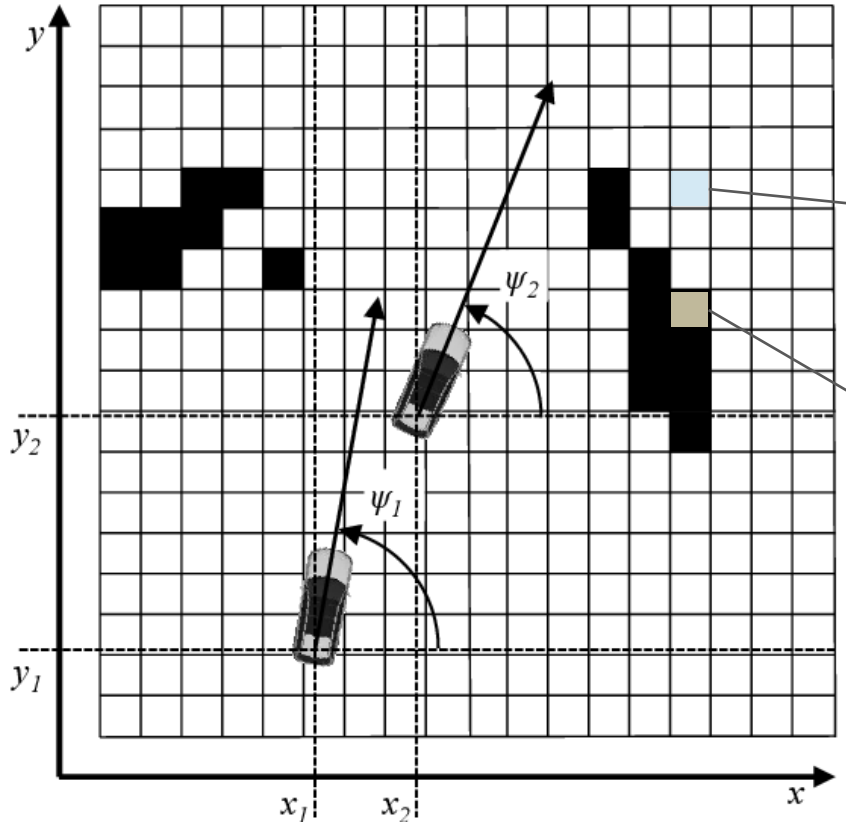
02 BEV map

BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers (ECCV 2022)



02 Occupancy map

Occupancy Map



[Driving in Occupancy Grid Map[1]]

Not Occupied → Traversable

Occupied → Non-Traversable

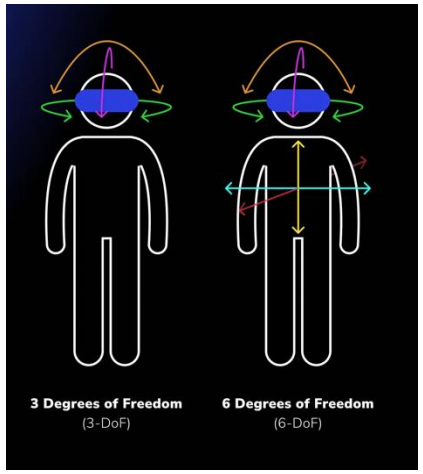
Pedestrian
Bicycle
Obstacle
...

[1] Nuß, D. S. (2016). *A Random Finite Set Approach for Dynamic Occupancy Grid Maps*.

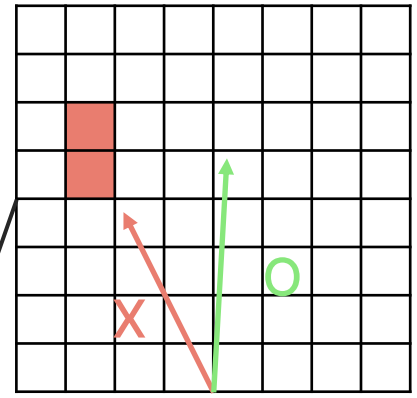
02 Occupancy map

Why 2D Occupancy maps?

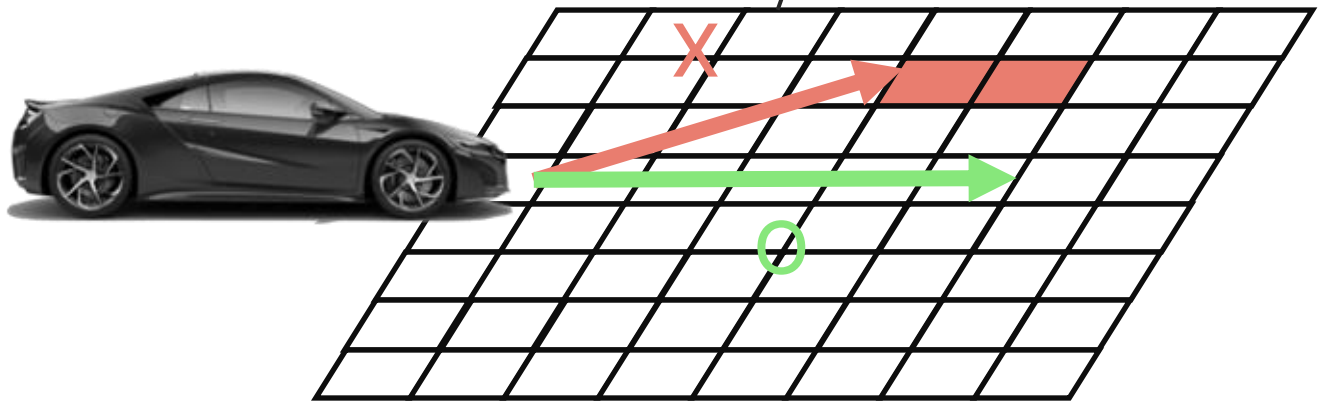
- Simplifies obstacle representation using approximate positions
  - Reduces computational complexity
- Only 3DOF is required for navigation
  - $(x, y, z, roll, pitch, yaw) \rightarrow (x, y, yaw)$
  - Flat-ground assumption (no uneven terrain or slopes in this project)



[3-DOF vs. 6-DOF [1]]



[Top-Down View]



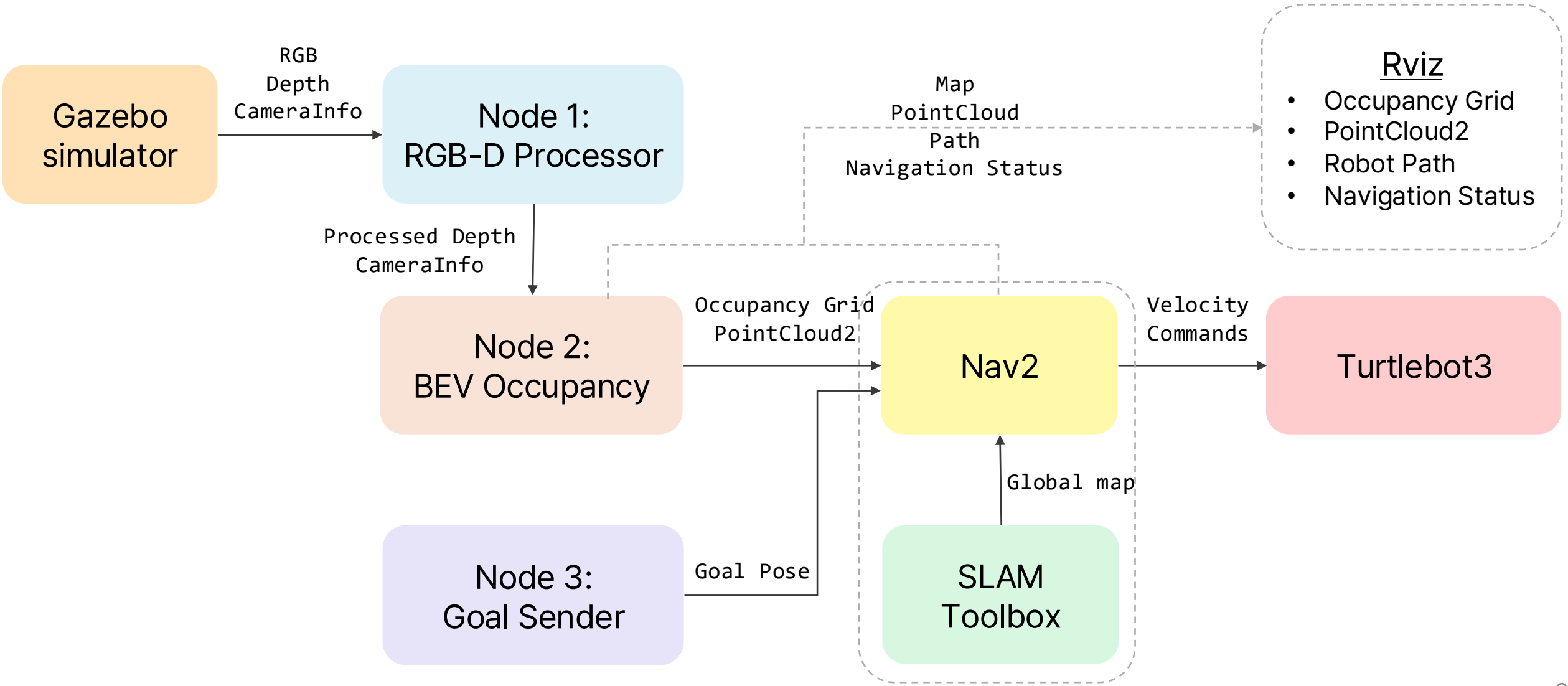
[2D Occupancy Map]

[1] 6DoF vs. 3DoF: Degrees of freedom in VR explained

03. System Architecture

03 Overall System Architecture

- Composed with Gazebo, 3 ROS2 nodes, SLAM Toolbox, Nav2, RViz2



03 Nodes

1. RGB-D Processor Node

- Role: Synchronizes RGB-D camera data and preprocesses depth images
- Input
  - RGB image, Depth image
  - Camera information
- Output
  - Processed RGB image, Processed Depth image
  - Camera information
- Functions
  - Synchronize RGB, depth, and camera information using ApproximateTimeSynchronizer
  - Filter invalid depth values
    - Nan and Inf values
    - 0.1m < Depth < 10.0m
  - Replace invalid depth values with zero

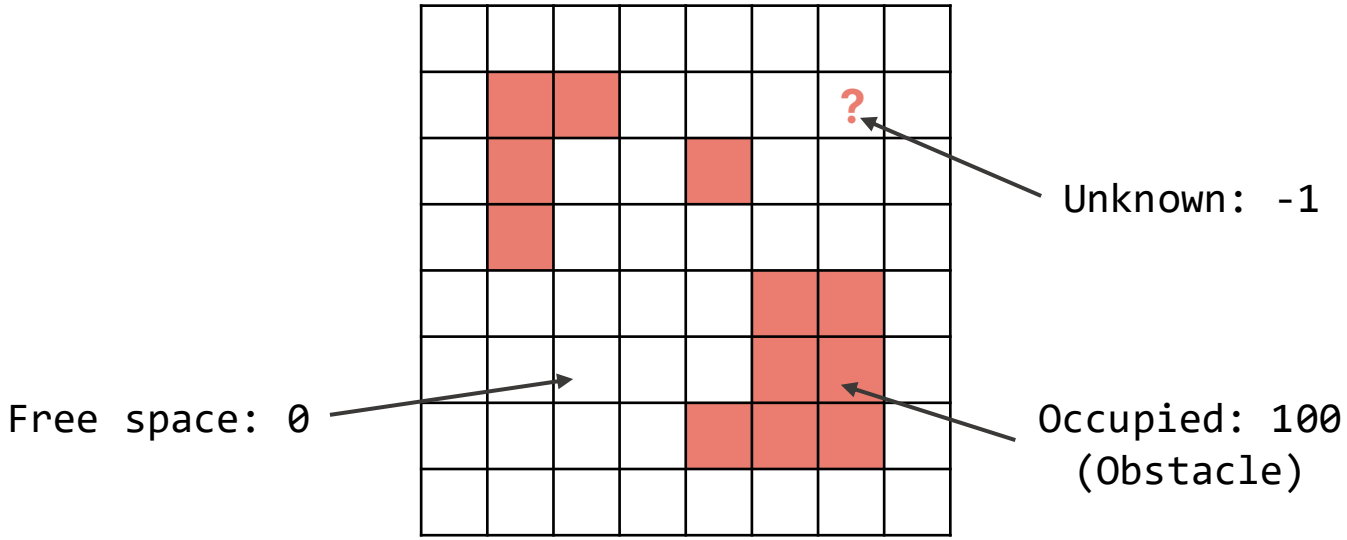
03 Nodes

2. BEV Occupancy Node

- Role: Converts preprocessed depth images into a BEV occupancy map
- Input
  - Processed depth, camera info
- Output
  - /bev_map/occupancy
  - /bev_map/pointcloud
- Processing pipeline
  1. Depth image
  2. Pinhole back-projection
  3. 3D point cloud
  4. Coordinate transformation
  5. Height filtering
  6. BEV occupancy grid

Parameters

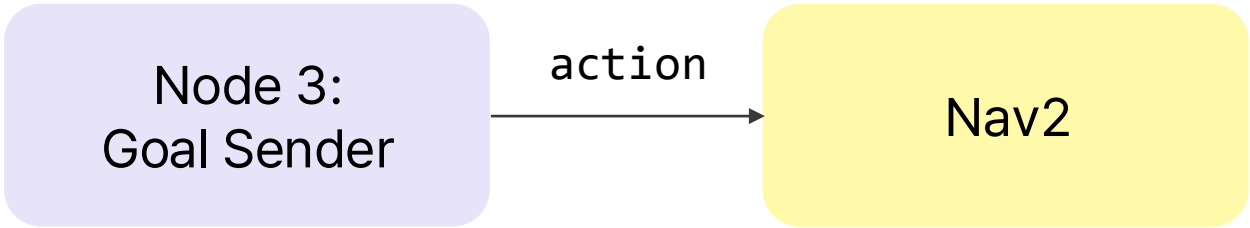
- Resolution: 0.05m
- Grid size: 10m x 10m
- Camera height: 0.3m
- Camera pitch: 15°
- Obstacle height: 0.05~2.0m



03 Nodes

3. Goal Sender Node

- Role:
  - Sends navigation goals to Nav2
  - Monitors navigation status
- Input
  - /bev_map/occupancy
- Action Client
  - /navigate_to_pose
- Output
  - /navigation_status
- Functions
  - Send target pose to Nav2
  - Monitor navigation process
  - Report navigation status



03 Nav2

Integration with Nav2

- 2 types of costmaps are used:
  1. Global costmap
     - Based on the `/map` generated by SLAM Toolbox
     - Used for global path planning from the robot to the goal
  2. Local costmap
     - Based on `/bev_map/pointcloud` published by the BEV Occupancy Node
     - Reflects real-time obstacles around the robot
     - PointCloud2 is used as input to the Nav2 ObstacleLayer
- The BEV OccupancyGrid is mainly used for visualization
- PointCloud2 is used for integration with Nav2 through the ObstacleLayer

03 Coordinate Transformation

- Depth images are converted into 3D points using the camera intrinsic parameters
- The camera coordinate system is transformed into the robot coordinate system
- The transformed points are classified according to their height

Pinhole Back Projection → Pitch → Camera → Robot →

$$X_{cam} = \frac{(u - c_x) \cdot d}{f_x}$$

$$Y_{cam} = \frac{(v - c_y) \cdot d}{f_y}$$

$$Z_{cam} = d$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\theta & sin\theta \\ 0 & -sin\theta & cos\theta \end{bmatrix}$$

$$\theta = 15°$$

$$P_{corrected} = R_x(-pitch) \cdot P_{cam}$$

$$R_{axis} = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

$$P_{robot} = R_{axis} \cdot P_{corrected}$$

03 Coordinate Transformation

- Depth images are converted into 3D points using the camera intrinsic parameters
- The camera coordinate system is transformed into the robot coordinate system
- The transformed points are classified according to their height



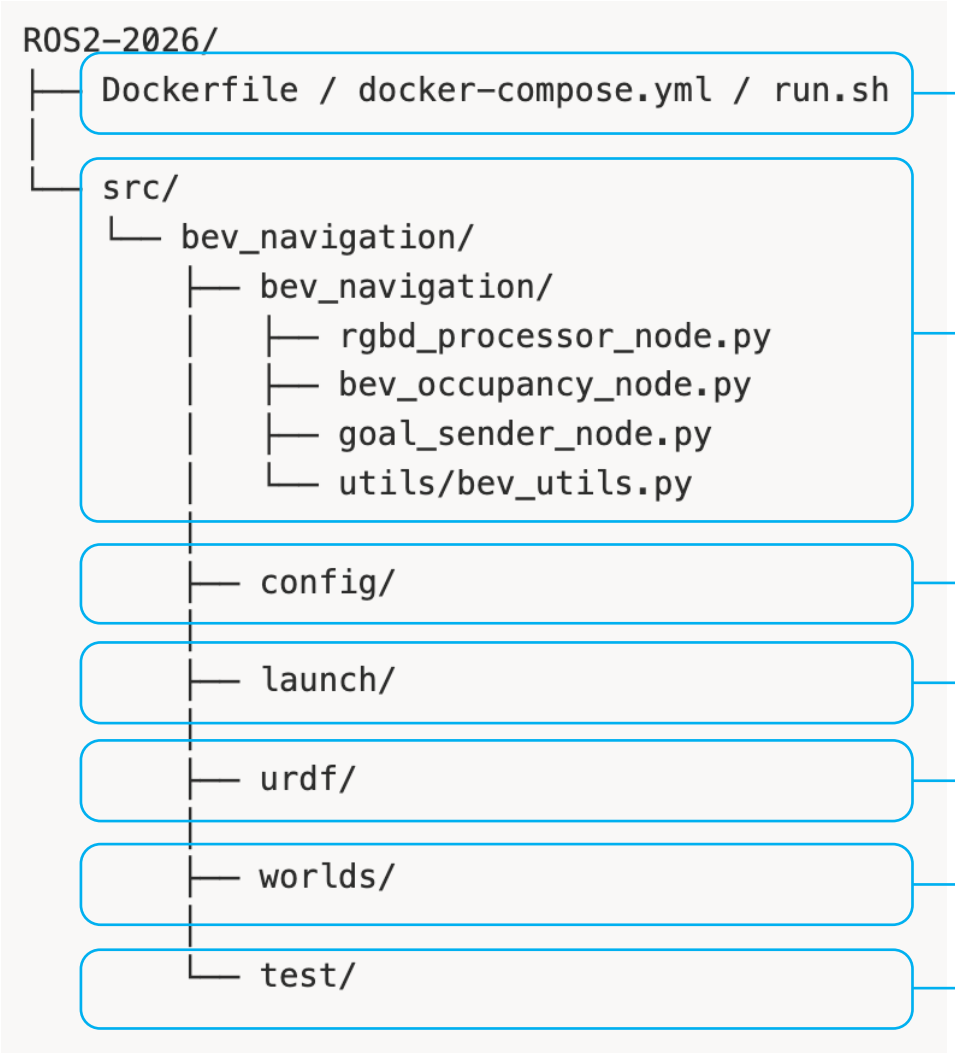
$$P^z_{robot} = P^z_{robot} + h_{camera}$$
$$h_{camera} = 0.3m$$

$$0.05m \leq P^z_{robot} \leq 2.0m$$

$$col = \frac{P^x_{robot} + offset}{resolution}$$

$$row = \frac{(P^y_{robot} + offset)}{resolution}$$

03 Project structure



- Environment setup and Docker-based execution scripts
- ROS2 nodes and core BEV transformation logic
- Parameters, Nav2 configuration, and RViz settings
- Launches Gazebo, Nav2, and all nodes together
- TurtleBot3 model equipped with an RGB-D camera
- Gazebo simulation environments
- Unit tests for BEV transformation utilities

03 Summary

Project Overview

- Reconstructs depth images into 3D spatial information
- Converts 3D points into a robot-centered BEV occupancy map
- Integrates the BEV map with the Nav2 costmap
- Enables autonomous navigation with real-time obstacle avoidance

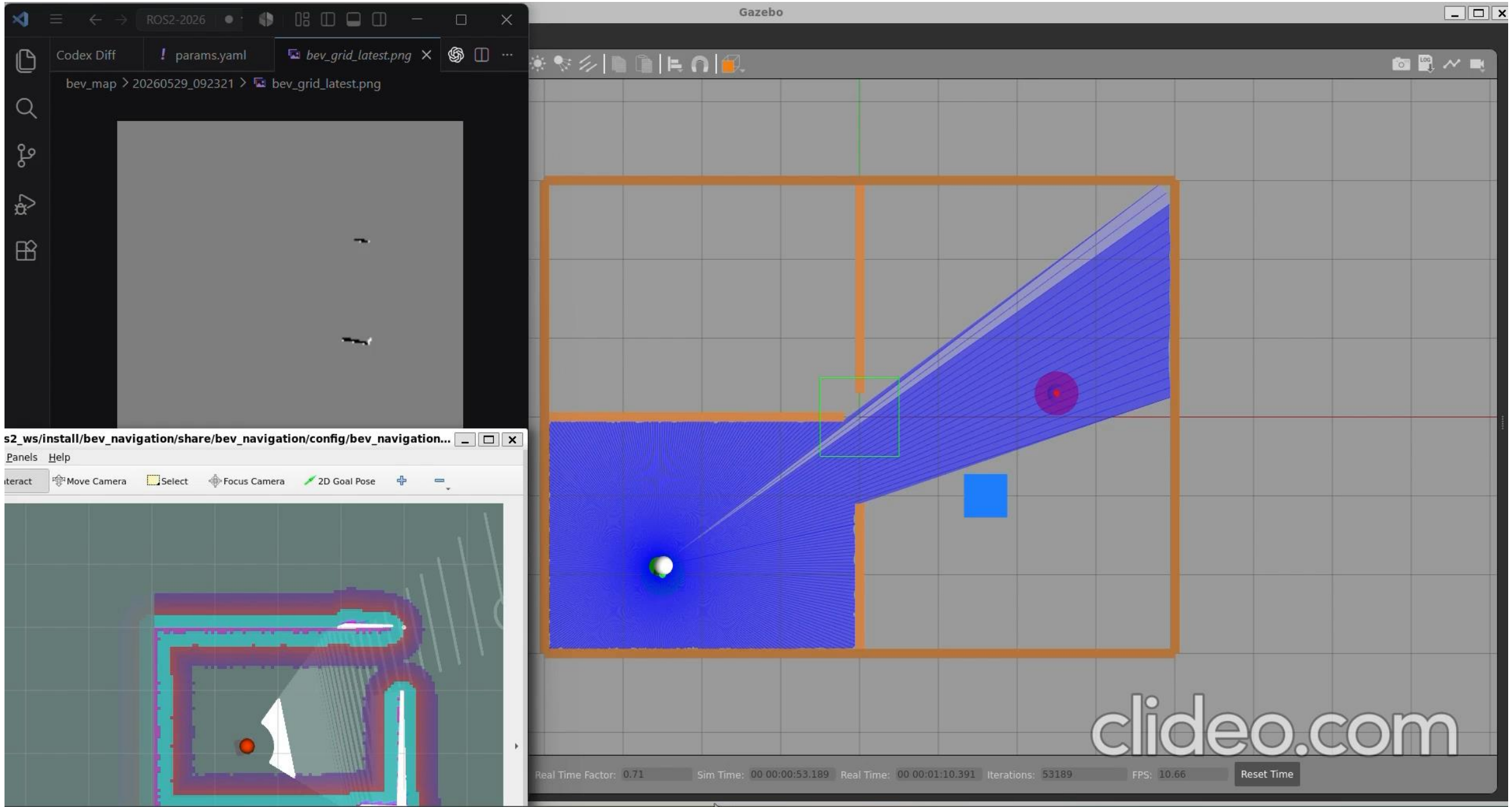
03 Summary

System Implementation

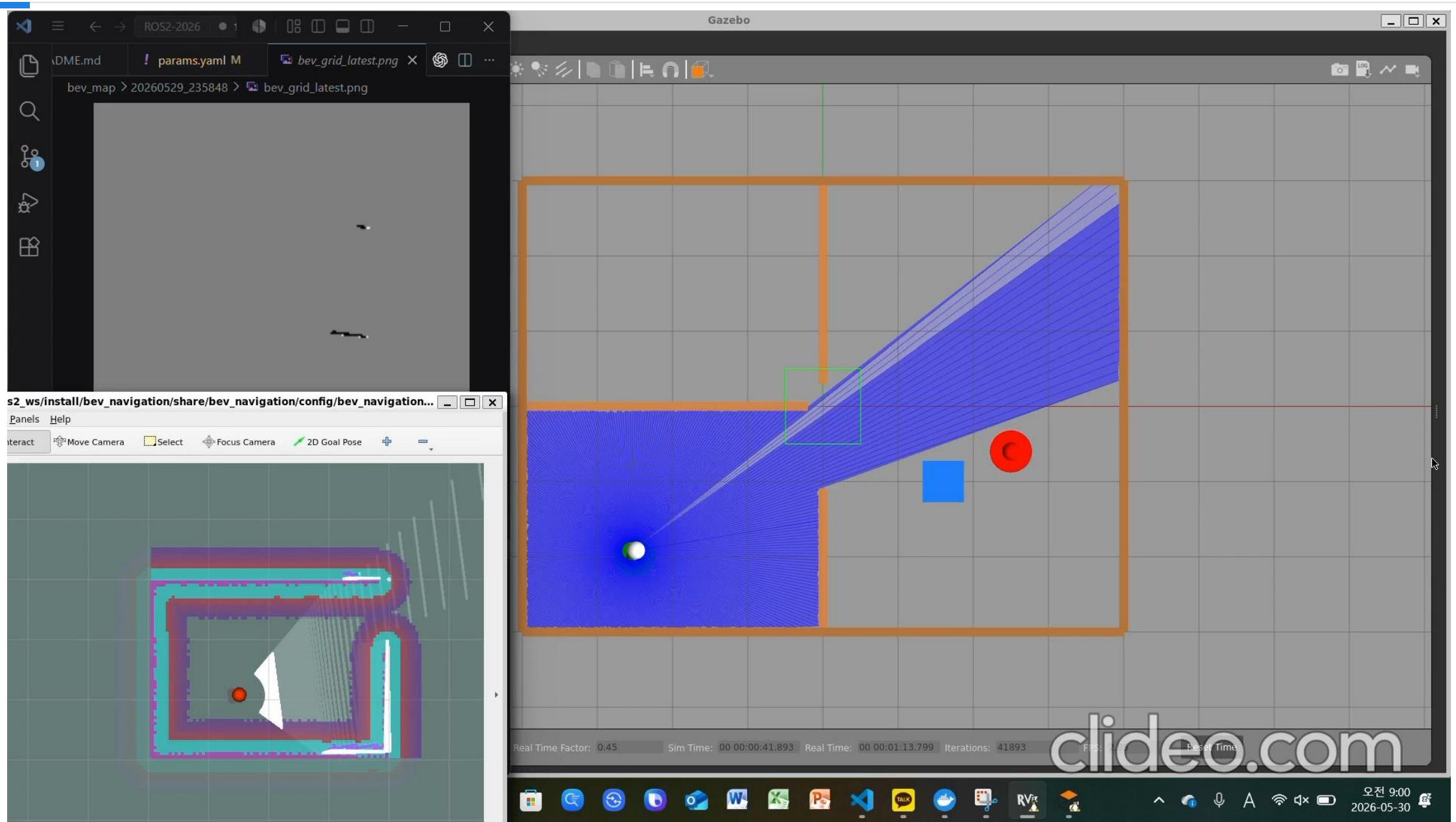
- ROS2 Humble
- Integrated components:
  - Gazebo simulator
  - TurtleBot3 Waffle
  - RGB-D Camera
  - SLAM Toolbox + Nav2 + RViz2
- Entire system is launched through a single launch file
- Mathematical transformations in `bev_utils.py` are verified using unit tests in `test_bev_utils.py`
- Codes are available at https://github.com/chehun16/ROS2-2026

04. Demo

04 Demo 1


ROS2-2026
Codex Diff
params.yaml
bev_grid_latest.png
bev_map > 20260529_092321 > bev_grid_latest.png
s2_ws/install/bev_navigation/share/bev_navigation/config/bev_navigation...
Panels Help
Move Camera
Select
Focus Camera
2D Goal Pose
Gazebo
Real Time Factor: 0.71
Sim Time: 00 00:00:53.189
Real Time: 00 00:01:10.391
Iterations: 53189
FPS: 10.66
Reset Time
clideo.com

04 Demo 2


ROS2-2026
params.yaml M
bev_grid_latest.png
bev_map > 20260529_235848 > bev_grid_latest.png
Gazebo
Panels Help
Move Camera
Select
Focus Camera
2D Goal Pose
Real Time Factor: 0.45
Sim Time: 00 00:00:41.893
Real Time: 00 00:01:13.799
Iterations: 41893
clideo.com
오전 9:00
2026-05-30

05. Challenges and Future work

05 Challenges and Future Work

Challenges

- Integration and tuning of ROS2, Gazebo, SLAM, Nav2
- Limited obstacle detection range
- Environmental constraints

Future work

- Improve camera calibration and BEV accuracy
- Validate the system in more diverse environments
- Increase navigation robustness and stability

Thank you